02

AULA 02 · Quarta, 12/08

Arquitetura de Servidores

Monolítico, microserviços e o time de servidores

Módulo · Arquitetura de Serviços e Dados

Prof.: Everton J Benedicto

ada

Cronograma das Aulas

01
Seg 10/08
Introdução ao Módulo e Docker

02
Qui 13/08
Arquitetura de Servidores

03
Sex 14/08
Apache e Nginx

04
Seg 17/08
Proxy Reverso e DNS

05
Qua 19/08
CI/CD e Ciclo de Vida

06
Sex 21/08
Cloud Computing

07
Seg 24/08
Arquitetura de Dados e Data Warehouse

08
Ter 25/08
Data Lake

09
Sex 26/08
Concluir atividades e preencher rubricas

COMBINADO DA TURMA

Por favor!

Bora participar!



Câmera ligada + mão levantada
= aula top!

INTERVALO · 16:30 BRT

Combinados da nossa aula

1. Câmeras ligadas.
2. Entrem no horário: início e intervalo.
3. Participem da aula.
4. Tirem dúvidas.
5. Levantem a mão para tirar dúvidas.

ARQUITETURA MONOLÍTICA

O que é um Monolito?

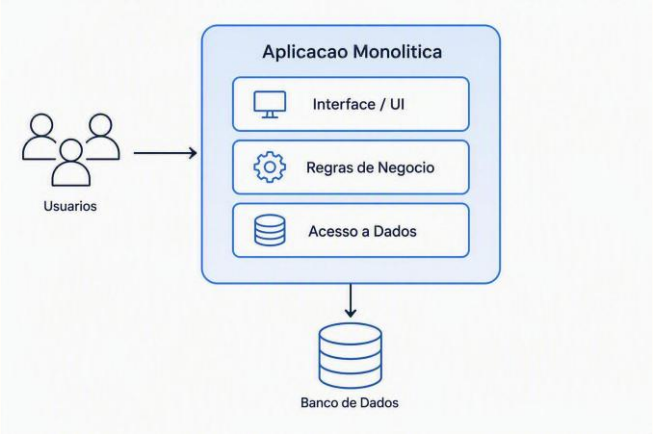
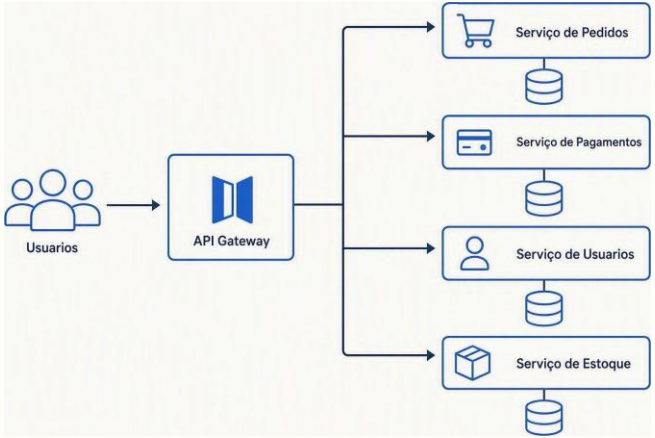


Diagrama ilustrativo · aplicação monolítica

1. **Base de código única:** modelo tradicional em que uma só aplicação executa várias funções de negócio
2. **Tudo acoplado:** interface, regras de negócio e acesso a dados no mesmo projeto e no mesmo deploy
3. **Vantagens:** simples de desenvolver, testar e implantar no início do projeto
4. **Desafios:** qualquer mudança exige novo deploy do sistema inteiro e a escala é do bloco todo

ARQUITETURA DE MICROSSERVIÇOS

O que são Microsserviços?



1. **Serviços autônomos:** uma coleção de pequenos serviços independentes, cada um com seu próprio ciclo de vida
2. **Contexto delimitado:** cada serviço implementa um único recurso de negócio e cuida dos próprios dados
3. **Comunicação por APIs:** os serviços conversam entre si por APIs, normalmente via um API Gateway
4. **Escala e deploy:** cada serviço evolui e escala sozinho, mas exige mais rede e monitoramento

Diagrama ilustrativo · arquitetura de microsserviços

CRITÉRIOS DE DECISÃO

Monolito ou Microsserviços?

Escolha Monolito quando...

- Time pequeno e produto no início
- Domínio ainda pouco definido
- Prazo curto e deploy simples
- Carga previsível e estável

Escolha Microsserviços quando...

- Times múltiplos e independentes
- Domínios de negócio bem definidos
- Partes com escala muito diferente
- Maturidade em DevOps e monitoramento

NA PRÁTICA

Migrando um Monolito: o caso do e-commerce

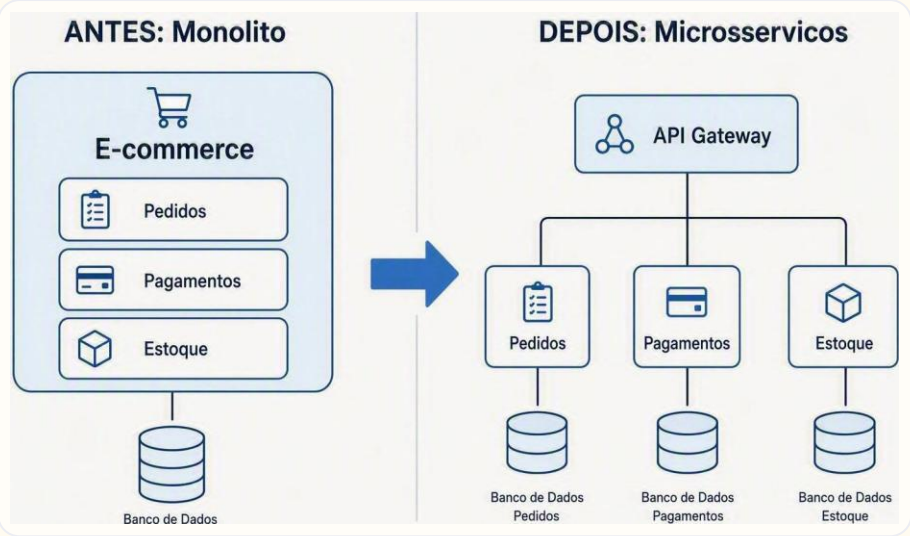
1. **Mapear o domínio:** identificar os contextos do sistema (pedidos, pagamentos, estoque, usuários)
2. **Priorizar a dor:** começar pelo módulo que mais trava o time ou mais escala
3. **Extrair o primeiro serviço:** tirar Pagamentos do monolito e expor por API
4. **Separar os dados:** cada serviço passa a ter o próprio banco, sem consultas cruzadas
5. **Rotear pelo gateway:** o API Gateway decide o que vai ao monolito e o que vai ao serviço novo
6. **Repetir e observar:** novos serviços aos poucos, com CI/CD, logs e monitoramento

💡 ESTRANGULAMENTO (STRANGLER FIG)

Nunca reescreva tudo de uma vez. O padrão Strangler Fig envolve o monolito com um gateway e substitui um pedaço de cada vez, até o sistema antigo deixar de ser usado.

NA PRÁTICA

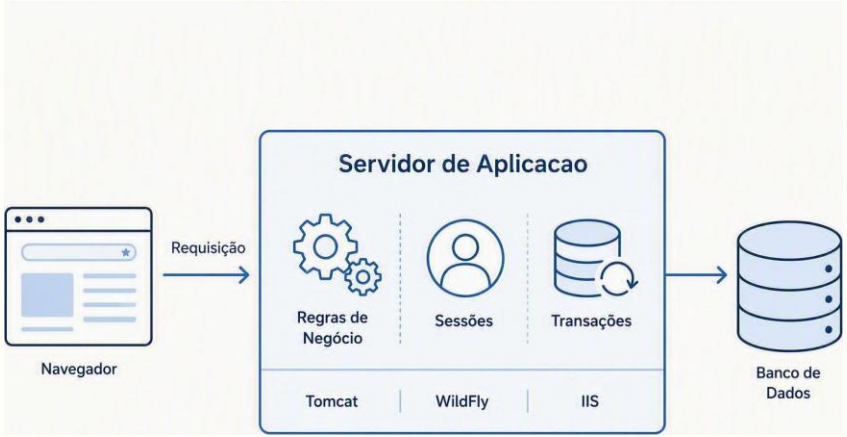
Antes e Depois da Migração



O monolito do e-commerce vira serviços independentes, cada um com o próprio banco, atrás de um API Gateway.

Diagrama ilustrativo · migração pelo padrão Strangler Fig

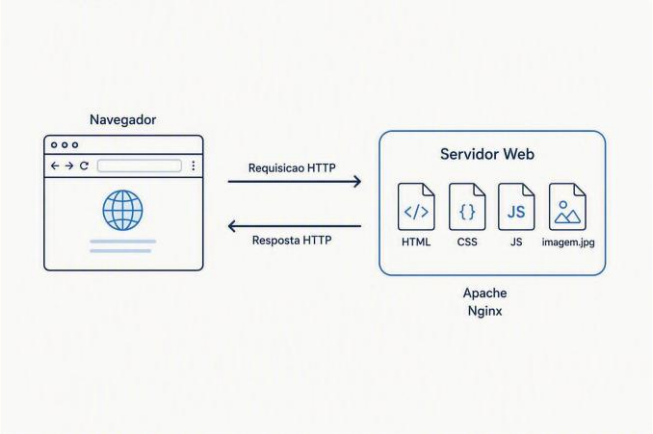
Servidor de Aplicação



1. **O que faz:** executa a lógica de negócio da aplicação e devolve conteúdo dinâmico
2. **Recursos:** gerencia sessões, transações, segurança e pool de conexões com o banco
3. **Exemplos:** Apache Tomcat, WildFly/JBoss, GlassFish, WebLogic e IIS
4. **Na prática:** recebe do servidor web o que exige processamento e consulta ao banco

Diagrama ilustrativo · o servidor de aplicação executa a lógica de negócio

Servidor Web



1. **O que faz:** recebe requisições HTTP e entrega o conteúdo estático do site
2. **Entrega:** arquivos HTML, CSS, JavaScript e imagens, direto do disco ou do cache
3. **Exemplos:** Apache HTTP Server, Nginx e IIS
4. **Na prática:** é a porta de entrada e repassa ao servidor de aplicação o que é dinâmico

Diagrama ilustrativo · o servidor web responde requisições HTTP

Servidor de Banco de Dados

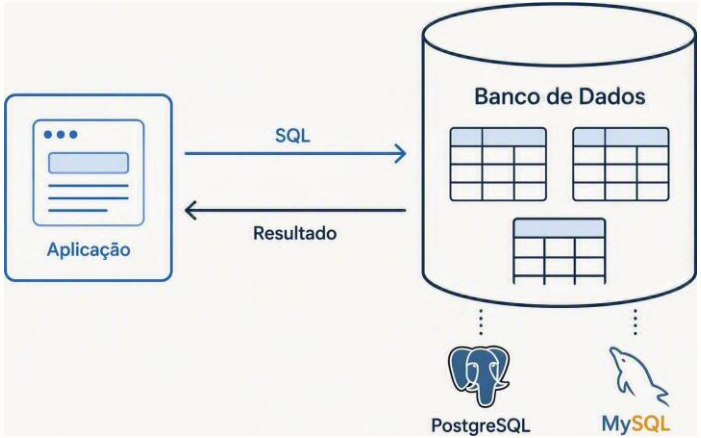


Diagrama ilustrativo · a aplicação consulta o banco e recebe os dados

1. **O que faz:** armazena, organiza e recupera os dados da aplicação com segurança
2. **Como responde:** recebe consultas (SQL) da aplicação e devolve apenas os dados pedidos
3. **Exemplos:** PostgreSQL, MySQL e SQL Server; MongoDB no mundo NoSQL
4. **Cuidados:** backup, índices e controle de acesso; costuma ser o gargalo do sistema

Servidor Proxy

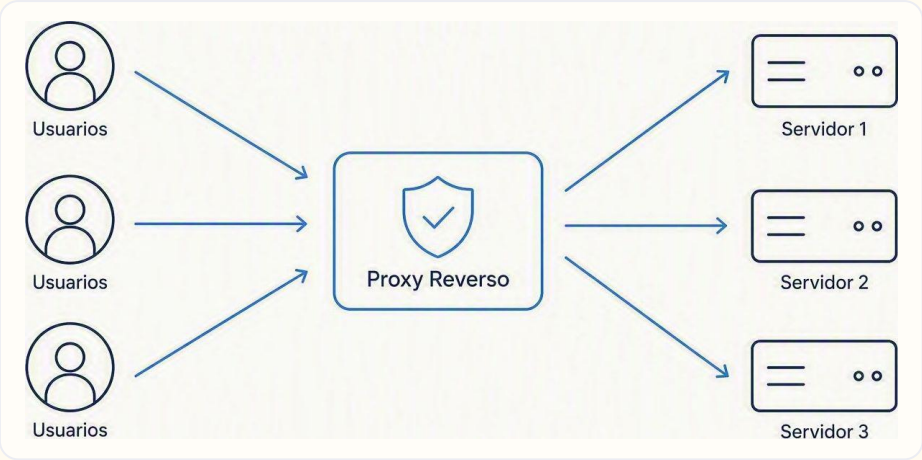
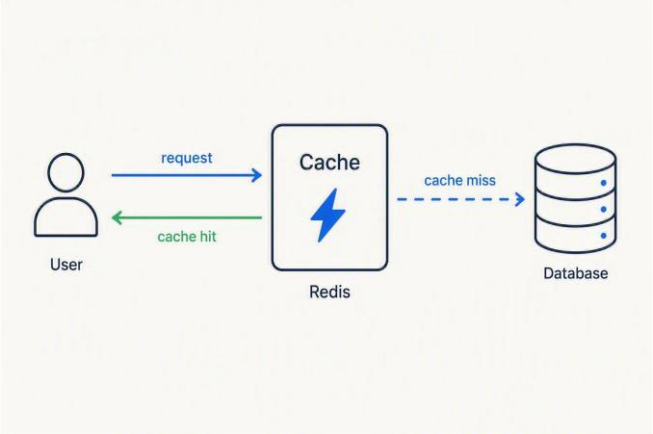


Diagrama ilustrativo · o proxy reverso distribui a carga entre os servidores

1. **O que faz:** fica na frente dos servidores e intermedia todas as requisições
2. **Balanceamento:** distribui a carga entre várias máquinas e desvia das que estão fora do ar
3. **Segurança:** esconde os servidores internos, encerra o HTTPS e filtra tráfego suspeito
4. **Exemplos:** Nginx, HAProxy e Traefik, tema aprofundado na próxima aula

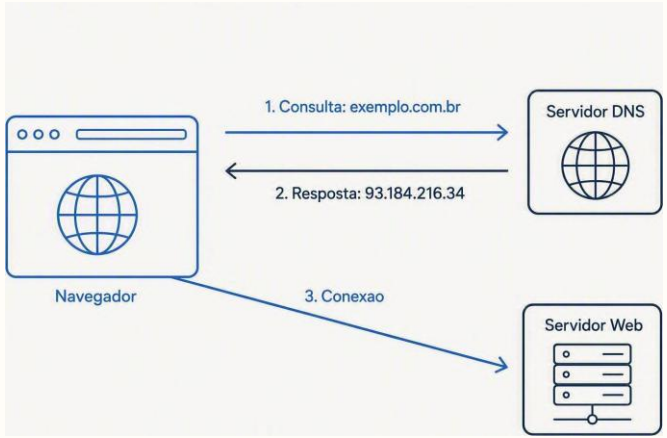
Servidor de Cache



1. **O que faz:** guarda em memória as respostas mais pedidas para devolvê-las na hora
2. **Cache hit e miss:** se o dado está no cache, responde direto; se não, busca no banco e guarda
3. **Ganhos:** respostas em milissegundos e muito menos carga no banco de dados
4. **Exemplos:** Redis e Memcached; exige definir o tempo de expiração dos dados

Diagrama ilustrativo · no cache hit a resposta vem da memória, sem ir ao banco

Servidor DNS



1. **O que faz:** traduz nomes de domínio em endereços IP, a agenda de contatos da internet
2. **Como funciona:** o navegador pergunta ao DNS o IP do domínio antes de abrir a conexão
3. **Registros:** A e AAAA apontam para IPs, CNAME cria apelidos e MX cuida do e-mail
4. **Na prática:** o cache do DNS acelera o acesso e o TTL define quando ele expira

Diagrama ilustrativo · o DNS traduz o nome do site no endereço IP do servidor

COMPONENTES

Tipos de Servidores — O Time Completo

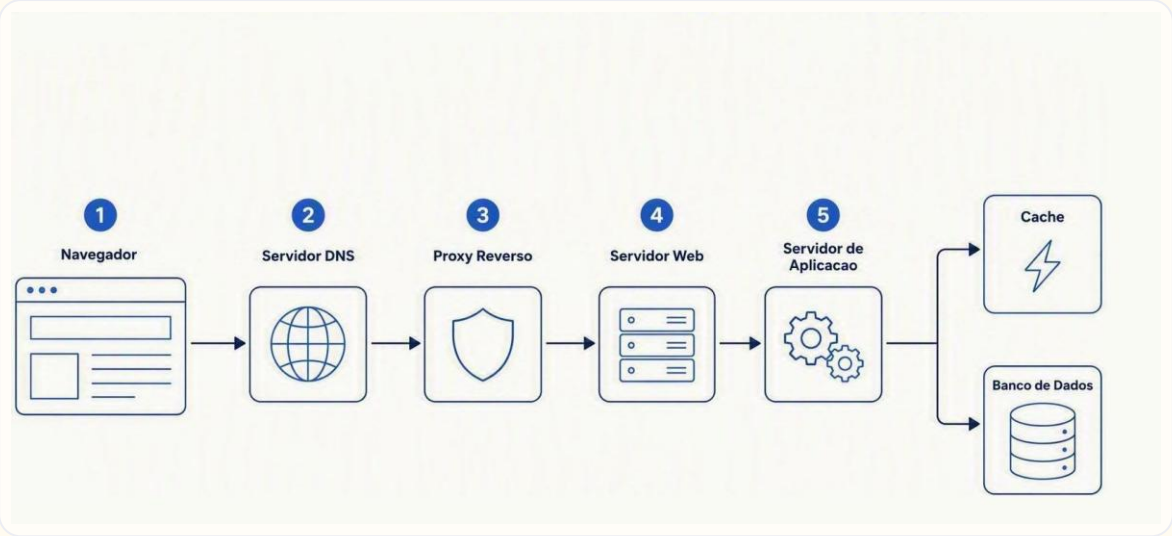
1. **Servidor de Aplicação:** executa a lógica de negócio (Tomcat, WildFly, GlassFish, WebLogic, IIS)
2. **Servidor Web:** entrega HTML, CSS e JS (Apache, Nginx, IIS)
3. **Banco de Dados:** armazena e recupera dados (PostgreSQL, MySQL)
4. **Servidor Proxy:** distribui carga e protege os servidores
5. **Servidor de Cache:** respostas rápidas da memória (Redis)
6. **Servidor DNS:** traduz nomes de domínio em IPs

💡 ANALOGIA

Um restaurante: o garçom (web) recebe o pedido, o chef (aplicação) cozinha, o estoque (banco) guarda os ingredientes e o caixa (proxy) organiza os pagamentos.

VISÃO GERAL

O Caminho Completo de uma Requisição



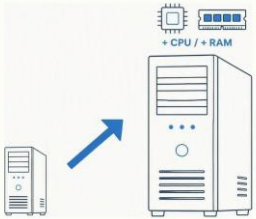
O navegador resolve o domínio no DNS, o proxy escolhe o servidor, o web entrega o estático, a aplicação processa a lógica e busca os dados no cache ou no banco.

Diagrama ilustrativo · cada peça da arquitetura no seu lugar

ESCALABILIDADE

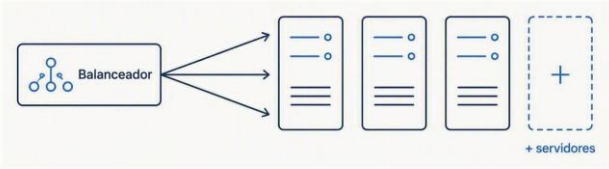
Como Crescer com Inteligência

Vertical (Scale-Up)



- Adicionar mais CPU/RAM ao servidor
- Simples de implementar
- Tem limite físico de crescimento
- Ponto único de falha

Horizontal (Scale-Out)



- Adicionar mais servidores ao pool
- Distribui a carga entre máquinas
- Flexível e tolerante a falhas
- Ideal para picos (Black Friday!)

DESAFIO

Mão na Massa: Sua Wiki em Container

- ✓ Criar uma imagem do Nginx ou Apache trazendo uma wiki sobre algum hobbie seu.
- ✓ Personalize sua index, traga imagens, suba o que achar necessário.
- ✓ Enviar para o Docker Hub com as devidas instruções.
- ✓ Entrega: link da imagem no Docker Hub + README com build e run.

RESUMO

Recapitulando a Aula

- ✓ Monolítico é simples no início; microserviços são flexíveis, porém mais complexos.
- ✓ Cada tipo de servidor tem uma função específica na arquitetura.
- ✓ Escalabilidade vertical (mais poder) vs horizontal (mais máquinas).
- ✓ Servidores de aplicação gerenciam transações, segurança e pools de conexão.